

SQL 是一种非过程化的查询语言，具有功能丰富、操作统一、语言简洁、易学易用等优点。但和程序设计语言相比，高度非过程化的特性造成了它的一个弱点：缺少流程控制能力，难以实现业务中的逻辑控制。数据库编程技术可以有效消除 SQL 无法实现复杂应用的问题，从而提高关系数据库管理系统支撑复杂应用的能力。

本章导读

本章从两个方面讲解数据库编程技术。一是扩展 SQL 自身的功能；二是在高级语言程序中使用 SQL，实现高级语言与数据库之间的交互，开发应用需求中复杂业务逻辑的技术和方法。

本章首先以实际应用任务需求为牵引，对 SQL 表达能力的限制进行说明，并阐述扩展 SQL 能力的技术途径。具体包括引入新的 SQL 子句，以增强 SQL 功能；引入新的内置函数；引入过程化 SQL (procedural language/SQL, PL/SQL) 和存储过程/存储函数的概念和技术等。

随后讲解 Java 数据库互连 (Java database connectivity, JDBC)，即在 Java 程序中如何访问数据库中的数据。最后讲解基于 MVC 框架的数据库应用开发，在实际的项目开发中，采用开发框架的方式可以提高项目开发的效率。

希望读者了解新的 SQL 子句，掌握 WITH RECURSIVE 子句的使用方法，提高 SQL 语句的可读性；掌握 PL/SQL、存储过程和存储函数等技术，以完成较为复杂的业务逻辑；能够根据实际应用场景，在第 7 章数据库设计的基础上进行应用开发，通过实践反复练习，掌握本章介绍的数据库编程方法和技术。

8.1 概 述

8.1.1 SQL 表达能力的限制

SQL 虽然功能强大，但是也有不足。本小节基于第 2 章的“学生选课”示例列举了 4 个较为

复杂的任务，这些任务直接使用基本的 SQL 语句都比较难以完成。

任务 1 打印“数据库系统概论”课程的所有先修课信息。

任务 1 的难点在于课程可能同时存在直接先修课和间接先修课。直接先修课的计算可以通过自身连接查询来完成，但间接先修课的计算则需要递归的查询。显然，任务 1 是一个递归的查询。其求解思路如下：

① 步骤 1。找出“数据库系统概论”课程的全部直接先修课，记为 $L[1]$ ；如果 $L[1]$ 为空，则任务 1 结束。

② 步骤 $i(i \geq 2)$ 。找出集合 $L[i-1]$ 中每一门课程的全部直接先修课，并计算它们的并集，记为 $L[i]$ 。

③ 迭代执行步骤 i ，直到并集 $L[i]$ 为空。输出 $L[1] \cup \dots \cup L[i]$ 。

在第 2 章表 2.1 所示的 Course 表中，执行步骤 1，找出“数据库系统概论”课程的直接先修课“数据结构”，即为 $L[1]$ 。执行步骤 2，找出“数据结构”课程的先修课“程序设计基础与 C 语言”，即为 $L[2]$ 。执行步骤 3，找出“程序设计基础与 C 语言”的先修课，得到 $L[3]$ 。可以发现 $L[3]$ 为空，递归查询结束。根据计算结果 $L[1] \cup L[2]$ ，任务 1 的输出如表 8.1 所示。

表 8.1 任务 1 的输出结果

Cpno	Cname
81002	数据结构
81001	程序设计基础与 C 语言

任务 1 可以使用扩展的 SQL 语句“WITH RECURSIVE 子句”完成，8.1.2 小节的例 8.2 将详细讲解。

任务 2 打印一周内将过生日的学生信息。

任务 2 需要数据库系统提供内置函数（本任务主要是日期函数）。其求解思路为：扫描每个学生的出生日期，例如 2000-6-12，并执行以下步骤：

① 把出生日期的年份换成当前日期所在的年份（例如 2021 年），即 2021-6-12。

② 获取当前系统的日期，例如 2021-6-9。

③ 确定过生日的日期范围 [2021-6-9, 2021-6-16]，即以当前日期为下界，当前日期后的第七天作为上界。

④ 判断出生日期是否在上述日期范围内，如果是，输出该学生信息。

8.1.2 小节的例 8.3 将讲解如何使用 SQL 的内置函数完成任务 2。

任务 3 给定学生学号，计算学生的平均学分绩点 GPA。

任务 3 需要用户自主设计业务处理逻辑。其求解思路为：给定学生学号，找出该学生所有选修课程的学分、成绩；根据每门课程的成绩，参照表 8.2 所示的“成绩和绩点对照表”，确定该成绩所处的范围，找出该门课程对应的绩点。

表 8.2 成绩和绩点对照表

编码	成绩下限	成绩上限	绩点
1	0	59	0
2	60	69	1
3	70	79	2
4	80	89	3
5	90	100	4

以学号为“20180001”的学生为例，该同学共选修了三门课程，如表 8.3 所示。

表 8.3 学号为“20180001”的学生的选修课程

学号 Sno	课程号 Cno	成绩 Grade	开课学期 Semester	教学班 Teachingclass
20180001	81001	85	20192	81001-01
20180001	81002	96	20201	81002-01
20180001	81003	87	20202	81003-01

课程号为“81001”的课程考试成绩为 85 分，参照表 8.2，该成绩对应的成绩范围为[80, 89]，绩点为 3；类似地，课程号为“81002”的课程考试成绩对应的绩点为 4，课程号为“81003”的课程考试成绩对应的绩点为 3。从 Course 表可知，这三门课程的学分都是 4。根据平均学分绩点的计算公式，即 $GPA = (\text{每门课程的学分} * \text{对应的绩点}) \text{的总和} / \text{学分的总和}$ ，得到该同学的 GPA 为 3.33。因此，上述查询结果为

GPA

3.33

8.2.5 小节的例 8.5 将介绍如何使用存储过程来完成任务 3。

任务 4 学生通过交互界面提交对某一位任课教师的教学评价意见，教师浏览这些评价意见并提供反馈信息。

任务 4 需要建立交互功能。一方面，学生需要找到指定的教学班和授课教师，建立如图 8.1 所示的交互界面并输入课程评价。另一方面，还要建立如图 8.2 所示的交互界面，教师浏览教学班学生的评价意见，并针对每条评价逐一做出回复。

8.3.3 小节的例 8.13 将讲解如何基于 JDBC 实现任务 4。

类似任务 1~任务 4 的应用还有很多，直接使用 SQL 实现都比较困难。那么如何扩展 SQL 的表达能力呢？主要有两大类途径：扩展 SQL 的功能和通过高级语言实现复杂应用。下面详

细介绍这两种途径。

课程评教	
<i>I</i>	信息
教学班	81001-01
课程	程序设计基础与C语言
任课老师	姜山
课程评价	总体上老师讲得很好。在数据库设计部分, 建议增加更丰富的应用实例 填写课程评价 填写完毕后点击添加 添加

图 8.1 学生输入并提交课程评价

查看教学班: 81001-01的学生评教		
学号	评论	操作
20180001	感谢老师	感谢认可
20180002	感谢老师	感谢认可
20180003	总体上老师讲得很好。在数据库设计部分, 建议增加更丰富的应用实例	回复

教师输入对学生评价的反馈 输入完毕后 点击回复

图 8.2 教师提交对学生评价的反馈

8.1.2 扩展 SQL 的功能

SQL 标准从发布以来随数据库技术的发展而不断丰富。通过扩展 SQL 可使其表达功能更为强大, 进而支持更为复杂的应用。SQL 扩展的方式主要包括引入新的 SQL 子句、引入新的内置函数, 以及引入 PL/SQL 与存储过程/存储函数。

1. 引入新的 SQL 子句

以任务 1 为例, SQL 标准引入了 WITH RECURSIVE 子句, 可执行递归查询。

类似于 WITH RECURSIVE 子句的 SQL 扩展还有很多, 例如面向联机分析处理的窗口子句, 面向空间数据管理、文档数据管理的 SQL 扩展等。

在介绍 WITH RECURSIVE 子句之前, 先了解 WITH 子句的用法。

(1) WITH 子句

WITH 子句用来创建一个命名的临时结果集, 该结果集仅在 SQL 语句(例如 SELECT,

INSERT或 DELETE)执行时有效,不长期存储。WITH 子句的引入,可以提高 SQL 语句的性能和可读性。

WITH 子句的语法格式如下:

```
WITH RS1[(<目标列>,<目标列>)]AS(SELECT 语句 1)[,
    RS2[(<目标列>,<目标列>)]AS(SELECT 语句 2),...]
SQL 语句;
```

RS1 为临时结果集的命名,RS1 对应 SELECT 语句 1 的执行结果,SELECT 语句 1 中的目标列与 RS1 中的目标列必须保持一致。

RS2 为临时结果集的命名,RS2 对应 SELECT 语句 2 的执行结果,SELECT 语句 2 中的目标列与 RS2 中的目标列必须保持一致。SQL 语句执行与 RS1,RS2...相关的查询。

[例 8.1]求 81001-01 和 81001-02 两个教学班之间学生选课平均成绩的差异。

```
WITH RS1(grade) AS
    (SELECT AVG(grade) FROM SC WHERE Teachingclass = '81001-01'),
    RS2(grade) AS
    (SELECT AVG(grade) FROM SC WHERE Teachingclass = '81001-02')
SELECT RS1.grade-RS2.grade FROM RS1, RS2
```

(2) WITH RECURSIVE 子句

WITH RECURSIVE 子句是 WITH 子句的一种特殊情况,用来查找具有层次结构的数据。WITH RECURSIVE 子句由两部分组成:第一部分是种子查询,用来初始化临时结果集,即第一层数据,假设记为 $L[1]$;第二部分是递归查询,该查询以 $L[1]$ 为基础,查找第二层数据 $L[2]$,然后以 $L[2]$ 为基础,查找第三层数据 $L[3]$,以此类推。递归查询需要设定每一层次数据的查找方式,比如查找第二层的数据时,需要用第一层数据中某个属性(或属性组)和关系表中的另外一个属性(或属性组)进行匹配,按照这个条件查找出来的数据就是第二层数据,同理查找第三层、第四层数据等。

WITH RECURSIVE 子句的语法格式如下:

```
WITH RECURSIVE RS AS
(
    SEED QUERY           /* 初始化查询的临时结果集,记为 L[1] */
    UNION [ALL]          /* 是否需要保留重复记录,加 ALL 为保留 */
    RECURSIVE QUERY      /* 执行递归查询,得到全部临时结果集,即 L[2] ∪ ... ∪ L[i] */
)
SQL 语句 /* 执行与 RS 相关的查询 */
```

[例 8.2] 打印“数据库系统概论”课程的所有先修课信息。

WITH RECURSIVE RS AS

(SELECT Cjno FROM Course WHERE Cname = '数据库系统概论'

/* 初始化 RS, 假设结果集为 L[1], 即“数据库系统概论”的所有直接先修课 */

UNION

SELECT Course.Cjno FROM Course, RS WHERE RS.Cjno = Course.Cno

) /* 递归查询第 i 层 ($i \geq 1$) 的数据, 即第 i-1 层数据的直接先修课课程号, 并更新 RS */

SELECT Cno, Cname FROM Course WHERE Cno IN (SELECT Cjno FROM RS);

/* 根据 RS 中记录的所有先修课程号, 通过查找课程表, 输出课程号与课程名 */

第2章图2.1所示的 Course 表中, 课程之间的先修课关系具有层次结构, 如图8.3所示。种子查询返回的是以“81003 数据库系统概论”课程为根的第一层数据, 即“数据库系统概论”课程的直接先修课 **Cjno=81002**, 这时 RS 被初始化为 81002。执行递归查询, RS 表与 Course 表做自然连接, 查找课程 Cno=81002 的先修课, 返回其先修课 **Cno=81001**, 这是第二层数据。用 RS 表中的第二层数据与 Course 表做自然连接, 查找课程 Cno=81001 的先修课, 这是第三层数据。因为 Cno=81001 的课程不存在直接先修课, 因此第三层数据为空。这时, 递归查询结束。“数据库系统概论”课程 81003 所有的先修课有 81002、81001。



图 8.3 课程之间的先修课关系具有明显的分层结构

注意：使用 WITH RECURSIVE 子句查找所有先修课课程信息，同样适用于一门课程存在多门先修课的场景。

2. 引入新的内置函数

关系数据库管理系统提供了丰富的内置函数，方便用户的操作。



扩展阅读：几个数据库产品的部分常用内置函数
(1)

常用的基本内置函数可以分为数学函数(如绝对值函数等)、聚合函数(如求和、求平均函数等)、字符串函数(如求字符串长度、求子串函数等)、日期和时间函数(如返回当前日期函数等)、格式化函数(如字符串转 IP 地址函数等)、控制流函数(如逻辑判断函数等)、加密函数(如使用密钥对字符串加密函数等)、系统信息函数(如返回当前数据库名、服务器版本函数)等。本书整理了几个数据库产品的部分常用内置函数,读者可以阅读二维码中的相关内容。



扩展阅读:几个数据库产品的部分常用内置函数
(2)

[例 8.3]打印一周内将过生日的学生信息。

```
SELECT Sno, Sname, Ssex, Sbirthdate, Smajor
FROM Student
WHERE to_date(to_char(current_date,'yyyy') || '-' || to_char(Sbirthdate,'mm-dd'))
BETWEEN current_date AND current_date + interval '7' day
```

在 WHERE 语句中使用了以下内置函数:

- ① 内置函数 `current_date` 返回当前的系统日期,例如 2021-6-9。
- ② 内置函数 `to_char(current_date, 'yyyy')` 返回当前系统日期的年份,例如 2021; `to_char(Sbirthdate, 'mm-dd')` 返回学生出生日期中具体的月份和日期,例如 6-9。
- ③ `to_date(to_char(current_date, 'yyyy') || '-' || to_char(Sbirthdate, 'mm-dd'))` 表示把当前年份与出生日期用 '-' 连在一起。符号 "||" 用于把其左右两边的字符串连在一起。
- ④ 内置函数 `to_date()` 的作用是将字符类型按一定格式转化为日期类型。
- ⑤ `current_date + interval '7' day` 是对当前的日期调整后的日期。参数 `interval` 是年(yyyy)、季度(q)、月(m)、日(d)、时(h)等粒度的时间单位。

例如, `current_date + interval '7' day` 获得当前日期之后第七天的日期,即返回 2021-6-16。

通过执行此 WHERE 语句,判断学生表中每位学生转换后的出生日期是否在 [2021-6-9, 2021-6-16] 区间内,如果是,打印该学生的信息。

3. 引入 PL/SQL 与存储过程/存储函数

内置函数是关系数据库管理系统默认创建的。此外,还可以通过安装数据库的扩展插件添加该插件自带的内置函数。但是,在实际应用中仅仅使用关系数据库管理系统默认的或扩展插件自带的内置函数还是不够的。因此,在关系数据库管理系统中引入了 PL/SQL、存储过程/存储函数等方法,使得用户可以自定义程序逻辑,开发完成业务逻辑复杂的应用系统。

例如,在 8.1.1 小节的任务 3 中,用户需要自定义计算平均学分绩点的函数。在该函数中引入逻辑判断和循环控制,逻辑判断用于获取每门课程成绩对应的绩点,循环控制用于计算课程总学分和总学分绩点,最终计算得到平均学分绩点。我们将在 8.2.5 小节的例 8.5 详细给出求解方法。

8.1.3 通过高级语言实现复杂应用

对于类似于 8.1.1 小节任务 4 的应用逻辑和功能需求,用户和数据库之间需要通过界面进行交互,这时可以通过高级语言(如 Java、C++、Python 等)访问数据库来解决。为此需要在关系数据库管理系统和应用系统之间建立交互接口,相互传递数据和控制信息,实现两类系统之间的互动。

作为各自独立的系统,关系数据库管理系统和应用系统之间的互操作是计算机软件发展中非常重要的一个话题。随着技术的进步和观念的更新,其交互方式也一直在丰富和发展,主要有以下几种方式。

1. 通过动态链接库调用的方式

在大型软件系统构建过程中,为了提高软件生产率,提出了“模块化”的概念,也就是将大型软件分解为一些小的模块,每个模块可以组织成“子程序”(或者“函数”),之后通过在一个程序中“调用”(CALL)另一个子程序来实现程序与子程序之间的交互。在这样的思路下,关系数据库管理系统的功能被包装成一个子程序,由应用程序通过动态链接库调用来获得数据管理(包括查询、管理等)的功能。

2. 基于嵌入式 SQL 的方式

嵌入式 SQL 是将 SQL 语句嵌入某程序设计语言,被嵌入的程序设计语言称为宿主语言,简称主语言。

对嵌入式 SQL,关系数据库管理系统一般采用预编译方法处理,即由其预编译程序对源程序进行扫描,识别出嵌入式 SQL 语句并将其转换成主语言的调用语句(对库函数的调用),再由主语言的编译程序将其编译成目标码。嵌入式 SQL 编程的好处在于把用户从复杂的函数调用中解放出来,用户只需要在主语言中使用 SQL 语句代替复杂的函数调用即可,从而较大幅度地简化了开发工作量。

将 SQL 嵌入高级语言中混合编程,SQL 语句负责操纵数据库,高级语言语句负责控制逻辑流程,这时程序中会含有两种不同计算模型的语句,它们之间应如何通信呢?

数据库工作单元与源程序工作单元之间的通信主要包括 SQL 通信区、主变量、游标等方式。

(1) SQL 通信区

SQL 通信区(SQL communication area, SQLCA)主要实现向主语言传递 SQL 语句的执行状态信息,使主语言能够据此信息控制程序流程。

SQL 语句执行后,系统要反馈给应用程序若干控制信息,主要包括描述数据库系统当前工作状态和运行环境的各种数据。这些信息将被送到 SQLCA 中,应用程序从 SQLCA 中取出这些状态信息,据此决定接下来执行的语句。

SQLCA 由关系数据库管理系统预先定义好,根据高级语言的不同有所不同。SQLCA 将作为高级语言变量定义的一部分而存在。

例如, SQLCA 中有一个变量 SQLCODE, 用来存放每次执行 SQL 语句后返回的代码。应用程序每执行完一条 SQL 语句之后都应测试一下 SQLCODE 值, 以了解该 SQL 语句是否执行成功。如果 SQL 语句执行不成功, 则在 SQLCODE 中存放的就是错误代码, 程序员可以根据错误代码查找问题, 应用程序也可以根据错误代码执行相应的处理。

(2) 主变量

主语言主要用主变量(host variable)向 SQL 语句提供参数。

嵌入式 SQL 语句中可以使用主语言的程序变量来输入或输出数据, 这些程序变量简称为**主变量**。主变量根据其作用的不同可分为输入主变量和输出主变量。**输入主变量**由应用程序对其赋值, SQL 语句引用;**输出主变量**由 SQL 语句对其赋值或设置状态信息, 返回给应用程序。

一个主变量可以附带一个任选的指示变量(indicator variable)。**指示变量**是一个整型变量, 用来“指示”所指主变量的值或条件。指示变量可以指示输入主变量是否为空值, 可以检测输出主变量是否为空值、值是否被截断等。

所有主变量和指示变量必须在 SQL 语句“BEGIN DECLARE SECTION”与“END DECLARE SECTION”之间进行说明。说明之后, 主变量可以在 SQL 语句中任何一个能够使用表达式的地方出现, 为了与数据库对象名(表名、视图名、列名等)区别, SQL 语句中的主变量名和指示变量名前要加冒号(:)作为标志。

(3) 游标

SQL 是面向集合的, 一条 SQL 语句可以产生或处理多条记录; 而高级语言是面向记录的, 一次只能处理一条记录。所以在 SQL 语句向应用程序输出数据时, 需要有一个内存区域来缓存数据, 这个区域就称为游标。用游标来协调这两种不同的处理方式。

游标是关系数据库管理系统为应用程序开设的一个数据缓冲区, 存放 SQL 语句的执行结果, 每个游标区都有一个名字和一个指针。应用程序可以通过游标指针逐一获取记录并进行处理。

3. 基于 ODBC/JDBC 的中间件方式

目前广泛使用的关系数据库管理系统有多种, 尽管这些系统都遵循 SQL 标准, 但是不同的系统有许多差异。因此, 在某个关系数据库管理系统下编写的应用程序并不能在另一个关系数据库管理系统下运行, 适应性和可移植性较差。例如, 运行在 Oracle 上的应用系统要在 KingbaseES 上运行, 就必须进行修改移植。这种修改移植比较烦琐, 开发人员必须清楚地了解不同关系数据库管理系统的区别, 细心地一一进行修改、测试。更重要的是, 许多应用程序需要共享多个部门的数据资源, 访问不同的关系数据库管理系统甚至文件系统。为此, 人们开始研究和开发连接不同关系数据库管理系统的方法、技术和软件, 使数据库系统“开放”, 能够实现数据库互连。

微软公司提出了 Windows 开放服务体系结构(Windows open services architecture, WOSA)的概念, 其核心思想是将微软平台上所有独立的软件系统都作为“设备”统一看待并标准化(如同打印机这类硬件设备)。设备的生产厂商按照标准提供驱动程序, 这样微软平台上的应用程序

就可以按照统一的方式访问不同生产厂商的设备了。应用程序使用某个具体设备之前只需安装该设备的驱动程序,并按照标准的接口方式使用即可,从而最大限度地规范了应用程序的开发。具体到数据库的场景,就是**开放数据库互连**(open database connectivity, ODBC)。

ODBC 是微软公司 WOSA 中有关数据库的一个组件,它建立了一组规范,并提供一组数据库应用编程接口(database application programming interface, DBAPI)。这样,无论使用什么数据库,都采用同样的一组编程接口来访问数据库,规范了应用程序的开发,提高了应用程序的可移植性。

数据库系统需要各自实现这组编程接口,形成该数据库的驱动程序。利用 ODBC 访问数据库之前需要安装数据库驱动程序, WOSA 中有一个数据库驱动程序管理器,负责管理各种数据库的驱动程序。

利用 ODBC 访问数据库的流程如下:

- ① 环境准备。配置数据源,即某个数据库,包含数据库名称,访问地址,用户名,口令等信息。需要用驱动程序管理器工具来完成。
- ② 设置环境。利用编程接口中的环境设置语句进行设置。
- ③ 进行数据库连接。
- ④ 完成相关操作。执行 SQL 语句,完成对数据库的相关操作、对返回结果集进行计算等。
- ⑤ 关闭数据库连接,释放占用的资源。

类似地,在微软之外的平台上,也有其他的一些机制,例如 JDBC(适用于 Java 环境)等。由于 Java 语言使用的普遍性,本书将在 8.3 节详细介绍如何使用 JDBC 访问数据库中的数据。

软件的体系结构在不断发展变化,访问数据库的方式也会随之发生变化,该过程不会终止。期待有更便捷高效的应用程序与数据库之间的交互方式的出现。

8.2 过程化 SQL

SQL 99 标准支持存储过程和函数的概念,商用关系数据库管理系统大多提供过程化的 SQL,如 Oracle 的 PL/SQL、Microsoft SQL Server 的 Transact-SQL、IBM DB2 的 SQL PL、Kingbase 的 PL/SQL 等。本节主要介绍过程化 SQL 的块结构、变量和常量的定义、流程控制、游标的定义与使用、存储过程及存储函数等内容。

8.2.1 过程化 SQL 的块结构

基本 SQL 是高度非过程化的语言,而过程化 SQL 是对基本 SQL 的扩展,在其基础上增加了一些描述过程控制的语句。

过程化 SQL 程序的基本结构是块,每个块可以包含定义部分和执行部分,块与块之间可以相互嵌套,每个块完成一个逻辑操作。图 8.4 是过程化 SQL 块的基本结构。

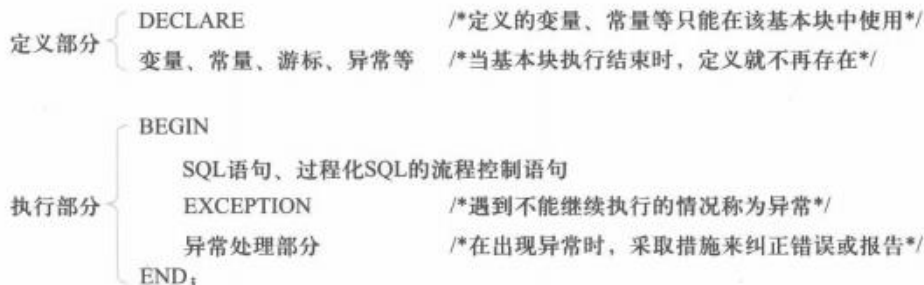


图 8.4 过程化 SQL 块的基本结构

8.2.2 变量和常量的定义

变量的定义格式如下:

变量名 数据类型 [[NOT NULL] := 初值表达式]

或

变量名 数据类型 [[NOT NULL] 初值表达式]

常量的定义格式如下:

常量名 数据类型 CONSTANT := 常量表达式

常量必须要赋予一个值, 并且该值在存在期间或常量的作用域内不能改变。如果试图修改它, 过程化 SQL 将返回一个异常。

赋值语句的格式如下:

变量名 := 表达式

8.2.3 流程控制

过程化 SQL 提供了流程控制语句, 主要有条件控制语句和循环控制语句。这些语句的语法、语义和一般的高级语言(如 C 语言)类似, 这里只做概要介绍。读者使用时需参考具体产品手册的语法规则。

1. 条件控制语句

条件控制语句一般包含三种形式的 IF 语句: IF-THEN 语句、IF-THEN-ELSE 语句和嵌套的 IF 语句。

① IF-THEN 语句的格式如下:

```
IF condition THEN
    Sequence_of_statements;    /* 条件为真时语句序列才被执行 */
END IF; /* 条件为假或 NULL 时什么也不做,控制转移至下一个语句 */
```

② IF-THEN-ELSE 语句的格式如下:

```
IF condition THEN
    Sequence_of_statements1;    /* 条件为真时执行语句序列 1 */
ELSE
    Sequence_of_statements2;    /* 条件为假或 NULL 时执行语句序列 2 */
END IF;
```

③ 嵌套的 IF 语句。在 THEN 和 ELSE 子句中还可以再包含 IF 语句,即 IF 语句可以嵌套。

2. 循环控制语句

过程化 SQL 有三种循环控制语句: LOOP 语句、WHILE-LOOP 语句和 FOR-LOOP 语句。

① 最简单的循环语句 LOOP 格式如下:

```
LOOP
    Sequence_of_statements;    /* 循环体,一组过程化 SQL 语句 */
END LOOP;
```

多数数据库服务器的过程化 SQL 都提供 EXIT、BREAK 或 LEAVE 等循环结束语句,以保证 LOOP 语句块能够在适当的条件下提前结束。

② WHILE-LOOP 循环语句的格式如下:

```
WHILE condition LOOP
    Sequence_of_statements;    /* 条件为真时执行循环体内的语句序列 */
END LOOP;
```

每次执行循环体语句之前首先要对条件进行求值,如果条件为真则执行循环体内的语句序列,如果条件为假则跳过循环并把控制传递给下一个语句。

③ FOR-LOOP 循环语句的格式如下:

```
FOR count IN [REVERSE] bound1 .. bound2 LOOP
    Sequence_of_statements;
END LOOP;
```

FOR 循环的基本执行过程是:将 count 设置为循环的下界 bound1,检查它是否小于上界 bound2。当指定 REVERSE 时则将 count 设置为循环的上界 bound2,检查 count 是否大于下界 bound1。如果越界则执行跳出循环,否则执行循环体,然后按照步长(+1 或-1)更新 count 的值,重新判断条件。

3. 错误处理

如果过程化 SQL 在执行时出现异常, 则应该让程序在产生异常的语句处停下来, 根据异常的类型执行异常处理语句。

SQL 标准对数据库服务器提供什么样的异常处理给出了建议, 要求过程化 SQL 管理器提供完善的异常处理机制。相对于嵌入式 SQL 简单地提供执行状态信息 SQLCODE, 这里的异常处理就复杂多了。读者要根据具体系统的支持情况进行错误处理。

8.2.4 游标的定义与使用

在过程化 SQL 中, 如果 SELECT 语句只返回一条记录, 可以将该结果存放到变量中。当查询返回多条记录时, 就要使用游标对结果集进行处理。一个游标与一个 SQL 语句相关联。

使用游标的步骤如下。

1. 声明游标

用 DECLARE 语句为一条 SELECT 语句定义游标:

```
DECLARE 游标名 [(参数1 数据类型, 参数2 数据类型, ...)] /* 不同的关系数据库产品语法稍有不同 */  
CURSOR FOR /* 在 Kingbase 中是"游标名 CURSOR FOR" */  
SELECT 语句; /* 在 Oracle 中是"CURSOR FOR 游标名" */
```

定义游标仅仅是一条说明性语句, 这时关系数据库管理系统并不执行 SELECT 语句。

2. 打开游标

用 OPEN 语句将定义的游标打开:

```
OPEN 游标名 [(参数1 数据类型, 参数2 数据类型, ...)];
```

打开游标实际上是执行相应的 SELECT 语句, 把查询结果取到缓冲区中。这时游标处于活动状态, 指针指向查询结果集中的第一条记录。

3. 使用游标

推进游标指针并取当前记录:

```
FETCH 游标名 INTO 变量1[, 变量2, ...];
```

其中变量必须与 SELECT 语句中的目标列表表达式具有一一对应关系。

用 FETCH 语句把游标指针向前推进一条记录, 同时将缓冲区中的当前记录取出送至变量供过程化 SQL 进一步处理。通过循环执行 FETCH 语句逐条取出结果集中的行进行处理。

4. 关闭游标

用 CLOSE 语句关闭游标, 释放结果集占用的缓冲区及其他资源。

```
CLOSE 游标名;
```

游标被关闭后就不再和原来的查询结果集相联系,但被关闭的游标可以再次被打开,与新的查询结果相联系。

[例 8.4] 根据给定学号 20180001, 使用游标输出该学生的全部选课记录。

```
DECLARE
    CnoOfStudent CHAR(10);
    GradeOfStudent INT;
    mycursor CURSOR FOR
    SELECT Cno, Grade FROM SC WHERE Sno='20180001';
BEGIN
    OPEN mycursor;    /* 打开游标 */
    LOOP              /* 循环遍历游标 */
        FETCH mycursor INTO CnoOfStudent, GradeOfStudent; /* 检索游标 */
        EXIT WHEN mycursor%NOTFOUND;
        RAISE NOTICE 'Sno:20180001, Cno:%, Grade:%', CnoOfStudent, GradeOfStudent;
    END LOOP;
    CLOSE mycursor;   /* 关闭游标 */
END;
```

8.2.5 存储过程

类似于高级语言程序,过程化 SQL 程序也可以被命名和编译并保存在数据库中,称为存储过程(stored procedure)或存储函数(stored function),供其他过程化 SQL 调用。存储过程或存储函数也是一类数据库的对象,需要有创建、删除等语句。

1. 创建存储过程

```
CREATE OR REPLACE PROCEDURE 过程名(
    [[IN|OUT|INOUT] 参数1 数据类型,
    [IN|OUT|INOUT] 参数2 数据类型,...]
) /* 存储过程首部 */
AS <过程化 SQL 块>; /* 存储过程体,描述该存储过程的操作 */
```

存储过程包括过程首部和过程体。在过程首部中,“过程名”是数据库服务器合法的对象标识;存储过程提供了 IN、OUT、INOUT 三种参数模式,分别对应输入、输出、输入输出三种语义,不声明参数模式时默认为 IN 类型。输入参数在被调用时需要指定参数值,输出参数调用时不传入参数值,而是作为返回值返回。输入输出参数调用时需要传入初始值,并会返回操作后的最终值。参数列表中需要指定参数模式、参数名以及参数的数据类型。

过程体是一个<过程化 SQL 块>,包括声明部分和可执行语句部分。<过程化 SQL 块>的基本结构已经在 8.2.1 小节中讲解了。

[例 8.5] 给定学生学号, 计算学生的平均学分绩点。

```
CREATE OR REPLACE PROCEDURE compGPA( /* 定义存储过程 compGPA */
    IN inSno CHAR(10); /* 输入参数: 学生学号 inSno */
    OUT outGPA FLOAT) /* 输出参数: 平均学分绩点 outGPA */
AS
DECLARE
    courseGPA INT; /* 声明变量 courseGPA, 临时存储课程绩点 */
    totalGPA INT; /* 声明变量 totalGPA, 临时存储总学分绩点 */
    totalCredit INT; /* 声明变量 totalCredit, 临时存储总学分 */
    grade INT; /* 声明变量 grade, 临时存储学生成绩 */
    credit INT; /* 声明变量 credit, 临时存储课程学分 */
    mycursor CURSOR FOR /* 声明游标 mycursor */
    SELECT Ccredit, grade FROM SC, Course
        WHERE Sno=inSno and SC. Cno=Course. Cno;
BEGIN
    totalGPA:=0;
    totalCredit:=0;
    OPEN mycursor; /* 打开游标 mycursor */
    LOOP /* 循环遍历游标 */
        FETCH mycursor INTO credit, grade; /* 检索游标 */
        EXIT WHEN mycursor%NOTFOUND;
        IF grade BETWEEN 90 AND 100 THEN courseGPA:=4.0;
        ELSEIF grade BETWEEN 80 AND 89 THEN courseGPA:=3.0;
        ELSEIF grade BETWEEN 70 AND 72 THEN courseGPA:=2.0;
        ELSEIF grade BETWEEN 60 AND 69 THEN courseGPA:=1.0;
        ELSE courseGPA:=0;
        END IF; /* 参照表 8.2, 根据成绩找出某门课程对应的绩点 */
        totalGPA:=totalGPA+courseGPA * credit;
        totalCredit:=totalCredit+credit;
    END LOOP;
    CLOSE mycursor; /* 关闭游标 mycursor */
    outGPA:=1.0 * totalGPA/totalCredit;
END;
```

2. 执行存储过程

CALL/PERFORM [PROCEDURE] 过程名([参数 1, 参数 2, ...]);

使用 CALL 或者 PERFORM 等方式激活存储过程的执行^①。在过程化 SQL 中, 数据库服务器支持在过程体中调用其他存储过程。

[例 8.6] 查询学号为 20180001 的学生的平均学分绩点 GPA。

```
DECLARE outGPA FLOAT;  
BEGIN  
    CALL compGPA('20180001',outGPA);  
    RAISE NOTICE 'GPA: %', outGPA;  
END;
```

在调用含有输入参数和输入输出参数的存储过程时, 需要指定具体的参数值。在调用含有输出参数的存储过程时, 对应位置不需要传入参数值, 但需要事先定义输出变量, 如例 8.6 所示。

3. 修改存储过程

可以使用 ALTER PROCEDURE 重命名一个存储过程, 格式如下:

```
ALTER PROCEDURE 过程名 1 RENAME TO 过程名 2;
```

可以使用 ALTER PROCEDURE 重新编译一个存储过程, 格式如下:

```
ALTER PROCEDURE 过程名 COMPILE;
```

4. 删除存储过程

```
DROP PROCEDURE 过程名;
```

5. 存储过程的优点

使用存储过程具有以下一些优点:

① 存储过程不像解释执行的 SQL 语句那样在提出操作请求时才进行语法分析和优化工作, 而是在创建存储过程时就完成了, 因而它的运行效率更高, 它提供了在服务器端快速执行 SQL 语句的有效途径。

② 存储过程降低了客户机和服务器之间的通信量。客户机上的应用程序只要通过网络向服务器发出调用存储过程的名字和参数, 就可以让关系数据库管理系统执行其中的多条 SQL 语句并进行数据处理。只有最终的处理结果才返回客户端。

③ 方便实施企业规则。可以把企业规则的运算程序写成存储过程放入数据库服务器中, 由关系数据库管理系统管理。这样做既有利于集中控制, 又能够方便地进行维护。当企业规则发生变化时只要修改存储过程即可, 无须修改其他应用程序。

8.2.6 存储函数

存储函数也称为自定义函数, 以区别于关系数据库管理系统的内置函数。存储函数和存储

^① 有些数据库管理系统, 如 Oracle、MySQL、Kingbase 等, 其 CALL 语句不含 PROCEDURE 关键字。

过程类似，都是持久性存储模块。存储函数的创建和存储过程也类似，不同之处是存储函数必须指定返回的类型。

1. 创建存储函数

```
CREATE OR REPLACE FUNCTION 函数名([参数 1 数据类型, 参数 2 数据类型, ...])  
RETURNS <类型>  
AS <过程化 SQL 块>;
```

2. 执行存储函数

```
CALL/SELECT 函数名([参数 1, 参数 2, ...]);
```

3. 修改存储函数

可以使用 ALTER FUNCTION 重命名一个存储函数，格式如下：

```
ALTER FUNCTION 函数名 1 RENAME TO 函数名 2;
```

可以使用 ALTER FUNCTION 重新编译一个存储函数，格式如下：

```
ALTER FUNCTION 函数名 COMPILE;
```

存储函数的概念与存储过程类似，这里就不再赘述了。有关示例可参见本章参考文献[4]。

8.3 JDBC 编程

Java 是一种面向对象的高级编程语言，具有健壮性、安全性、可移植性等优点。考虑到 Java 是最受欢迎的编程语言之一，全球有超过千万的开发者在使用它，本节将介绍如何进行 JDBC 编程，即如何使用 Java 语言调用 JDBC 来进行数据库应用程序的设计。

8.3.1 JDBC 工作原理概述

JDBC 应用系统的体系结构如图 8.5 所示，它由三部分构成：用户应用程序、JDBC 驱动程序管理器和数据源。

1. 用户应用程序

用户应用程序提供用户界面、应用逻辑和事务逻辑。使用 JDBC 开发数据库应用程序时，不用关心底层数据源是什么，只需要知道交互数据源的类型即可。用户应用程序与数据库中数据进行交互时，调用的是 JDBC 提供的标准接口(JDBC APIs)。JDBC 的标准接口主要包括：驱动程序管理接口(java.sql.DriverManager)、请求数据库连接时的连接接口(java.sql.Connection)、向数据源发送 SQL 语句并接收结果的执行接口(java.sql.Statement、java.sql.PreparedStatement、java.sql.CallableStatement)、数据库操作结果集处理接口(java.sql.ResultSet)等。

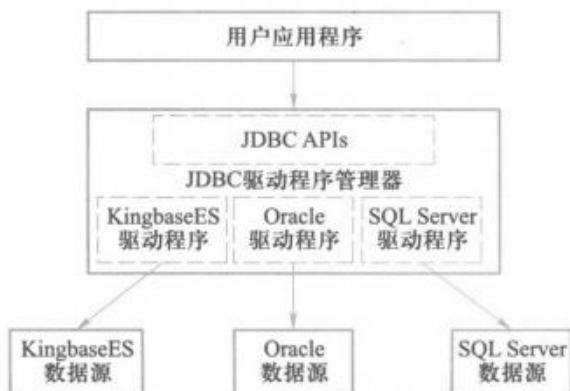


图 8.5 JDBC 应用系统的体系结构

2. JDBC 驱动程序管理器

JDBC 通过驱动程序管理器提供应用系统与数据库平台的独立性。

使用 JDBC 开发应用系统的程序简称为 JDBC 应用程序。JDBC 应用程序与数据库进行交互时不能直接存取数据，其各种操作请求需要通过调用 JDBC 驱动程序管理器提供的标准接口完成。JDBC 驱动程序管理器提供一系列标准接口，各个数据库厂商需要提供 JDBC 驱动程序，建立数据库访问接口与这些标准接口之间的映射。JDBC 应用程序对标准接口访问时，由驱动程序管理器提交给某个关系数据库管理系统的 JDBC 驱动程序，通过调用驱动程序所支持的函数来存取数据库中的数据。数据库的操作结果也通过驱动程序返回给应用程序。如果应用程序要操纵不同数据库管理系统下的数据库，就要动态地链接到不同的驱动程序上。

目前，JDBC 驱动程序主要有本地协议驱动和网络协议驱动两种。本地协议一般是数据源和应用程序在同一台机器上，JDBC 应用程序的接口调用将交由驱动程序直接完成对本地数据文件的存取操作，这时驱动程序相当于数据管理器。网络协议驱动支持客户-服务器、浏览器-服务器等网络环境下的数据访问，这时 JDBC 应用程序的接口调用将交由驱动程序访问网络中的中间件(应用服务器)，该中间件将 JDBC 调用直接或间接转换为特定数据库厂商提供的数据存取访问。

无论使用何种类型的驱动程序，驱动程序将完成数据库访问请求的提交和结果集接收，应用程序使用驱动程序提供的结果集管理接口，操纵执行后的结果数据。

3. 数据源

数据源是用户最终需要访问的数据，包含数据库位置、数据库类型、访问数据库的用户名和密码等信息，实际上是一种数据连接的抽象。应用程序与数据源是通过调用驱动管理器 DriverManager 的静态方法 getConnection(URL, UserName, Password) 建立连接的，参数分别为数据库的 URL、数据库用户名及密码。

8.3.2 JDBC APIs 基础

JDBC APIs 是各个数据库厂商提供的 JDBC 应用程序开发接口都要遵循的标准。

1. JDBC 中的常用类

使用 JDBC 进行应用程序开发涉及的所有类都包含在 `java.sql` 包中, 常用的有:

- ① 驱动程序类(`java.sql.Driver`), 由各数据库厂商提供。
- ② 驱动程序管理类(`java.sql.DriverManager`), 作用于应用程序与驱动程序之间。
- ③ 数据库连接类(`java.sql.Connection`), 用于建立与指定数据库的连接。
- ④ 静态 SQL 语句执行类(`java.sql.Statement`), 用于执行静态 SQL 语句并返回结果。
- ⑤ 动态 SQL 语句执行类(`java.sql.PreparedStatement`), 用于执行含参 SQL 语句并返回结果。

⑥ 存储过程语句执行类(`java.sql.CallableStatement`), 用于执行存储过程语句并返回结果。

- ⑦ 结果集处理类(`java.sql.ResultSet`), 用于检索结果集中的数据。

注意: JDBC 应用程序开发接口包含在上述 Java 类中, 不同 JDBC 版本上的接口名与接口的使用可能是有差异的, 读者必须注意使用的版本。

2. 数据类型

JDBC 定义了两套数据类型: SQL 数据类型和 Java 数据类型。SQL 数据类型用于数据源, 涉及例如创建关系模式时指定属性的数据类型。Java 数据类型用于应用程序, 涉及例如程序中定义的变量的数据类型。它们之间的转换规则如表 8.4 所示。当用户的应用程序通过 JDBC APIs 从数据库中存取数据时, Java 程序所使用的变量类型需要与数据库中关系模式属性的类型建立映射关系。例如, 通过结果集处理对象中的 `get` 方法从数据库中获得结果集, 并把记录中的属性值赋给应用程序变量; 为动态 SQL 语句执行对象绑定应用程序变量。

表 8.4 SQL 数据类型与 Java 数据类型之间的转换规则

数据类型	SQL 数据类型	Java 数据类型
SQL 数据类型	数据源之间转换	应用程序变量传递给语句参数
Java 数据类型	从数据库中读取数据 赋值给应用程序变量	应用程序变量之间转换

8.3.3 使用 JDBC 操纵数据库的工作流程

使用 JDBC 操纵数据库的工作流程如图 8.6 所示。本小节将结合具体的应用实例来介绍如何使用 JDBC 开发应用系统。



图 8.6 使用 JDBC 操纵数据库的工作流程

1. 加载驱动程序

在连接数据库之前，首先加载要连接数据库的驱动程序。可以使用 `java.lang.Class` 类的静态方法 `forName(String className)` 来加载指定数据库的驱动程序。一般来讲，在数据库厂商提供的 `Driver` 类中（例如 Kingbase 数据库的 `com.kingbase.Driver`），其静态代码段会默认包含一次驱动程序的注册操作。`forName` 方法在加载类时会自动执行一次该静态代码段，从而向驱动程序管理器注册一个驱动程序。

[例 8.7] 对 Kingbase、Oracle、SQL Server 数据库加载驱动。

```

Class.forName("com.kingbase.Driver");           /* Kingbase */
Class.forName("oracle.jdbc.OracleDriver");      /* Oracle */
Class.forName("com.microsoft.jdbc.sqlserver.SQLServerDriver"); /* SQL Server */
  
```

注意：在 JDBC 4.0 及以后版本不再需要使用 `Class.forName()` 显式地加载 JDBC 驱动程序。此外，根据 JDBC 开发规范，连接数据库等操作需要进行异常处理。本书为了叙述方便省略了异常处理，在实际的应用开发中建议参考 JDBC 开发规范补充异常处理逻辑，以增强应用系统的健壮性。

2. 建立与数据库的连接

(1) 定义连接的 URL

加载数据库驱动程序后，可以与数据库建立连接，连接需要通过 URL 地址定义。URL 地

址包含连接数据库所需的协议、子协议和数据库名称, 定义格式为

<协议名>:<子协议名>:<数据库名称>

JDBC 连接中的协议名总是 jdbc, 子协议名和数据库名称由具体连接的 DBMS 决定。

[例 8.8] 定义与 Kingbase、Oracle、SQL Server 数据库连接的 URL。

```
strURL="jdbc:kingbase://"+服务器地址+": "+端口号+"/"+数据库名;  
strURL="jdbc:oracle:thin:@ "+服务器地址+": "+端口号+": "+数据库名;  
strURL="jdbc:microsoft:sqlserver://"+服务器地址+": "+端口号+": "+数据库名;
```

Kingbase、Oracle、SQL Server 的默认端口号分别为 54321、1521、1433。

(2) 建立连接

利用生成的 URL 建立与数据库的连接。连接数据库时, 需要向 java.sql.DriverManager 类请求并创建一个数据库连接类(Connection)对象。使用 java.sql.DriverManager 类提供的静态方法 getConnection, 传入指定的连接 URL、数据库用户名和密码建立连接。

[例 8.9] 建立与 Kingbase 数据库的连接, 假定 Kingbase 服务器地址为 192.168.0.118, 端口号为 54321, 数据库名为 DB-Student, 用户名为 Info001, 密码为 123456。

```
String strURL="jdbc:kingbase://192.168.0.118:54321/DB-Student";  
Connection conn=DriverManager.getConnection(strURL,"Info001","123456");
```

3. 执行 SQL 语句

(1) 创建语句执行类对象

与数据库建立连接后, 就可以开始操作数据库了。JDBC 提供了语句执行类对象用来执行用户提交的 SQL 语句, 并接收 SQL 查询的结果集。

利用数据库连接类的 createStatement 方法可以创建静态语句执行类对象(Statement), 静态语句执行类对象只能用于执行静态的 SQL 语句。静态语句执行类还派生出两个动态语句执行类(PreparedStatement)和存储过程执行类(CallableStatement), 它们可以用于执行更高级的 SQL 语句。

动态语句执行类对象用来执行动态的 SQL 语句, 经过预编译, 可以有效防止 SQL 注入攻击, 从而提高数据库的安全性。另外, 预编译结果可以存储在动态语句执行类对象中, 多次执行该 SQL 语句时可以提高运行效率。

JDBC 还提供一个存储过程执行类, 专门用于执行数据库存储过程。

(2) 执行 SQL 语句

SQL 语句可以通过以下三种方法执行: executeQuery、executeUpdate 和 execute。

① executeQuery 用来执行数据库查询语句, 成功执行后返回一个 ResultSet 类(结果集)对象。

② executeUpdate 用来处理增加、删除、修改以及数据库定义语句, 成功执行后返回一个

int 类型值。

③ execute 用来处理存储过程或动态 SQL 语句, 成功执行后返回一个 boolean 类型值。

[例 8.10] 使用 JDBC 向课程评价表中插入一条记录。课程评价关系模式为 ClassAssess(Sno, Tno, TCno, Assess, CAtype Feedback)。

```
/* 创建 PreparedStatement 类对象 */
PreparedStatement stmt=conn. prepareStatement("INSERT INTO ClassAssess VALUES(?,?,?,?,?,?)");
/* 生成 PreparedStatement 类对象的动态参数,注意第六个字段 Feedback 未设置输入值 */
stmt.setString(1, "20180001");           /* 设置学生学号 */
stmt.setString(2, "19950018");           /* 设置职工号 */
stmt.setString(3, "81001-01");           /* 设置教学班号 */
stmt.setString(4, "老师讲得很出色");     /* 设置学生评价意见 */
stmt.setBoolean(5, true);                 /* 设置学生评价意见类型 */
stmt.executeUpdate();
```

[例 8.11] 利用例 8.5 创建的 compGPA 存储过程, 使用 JDBC 查询学号为 20180001 同学的平均学分绩点。

```
CallableStatement stmt=conn. prepareCall("{CALL compGPA (?,?)}");
/* 创建存储过程执行类对象 */
/* 生成 CallableStatement 类对象中的动态参数 */
stmt.setString(1, "20180001");           /* 设置输入参数 */
stmt.registerOutParameter(2, Types. REAL); /* 设置输出参数 */
stmt.execute();                           /* 执行存储过程 */
float fGPA=stmt. getFloat(2);              /* 获取输出参数 fGPA */
```

4. 处理结果集

使用 executeQuery 方法执行数据库查询语句, 成功执行后返回一个结果集类对象(ResultSet)。通过遍历结果集类对象中的每一个元组, 执行用户定义的应用逻辑。此外, 通过 ResultSet 类提供的 getXXX() 方法来获取结果集中元组的属性值。XXX 代表某种数据类型, 当用户想要获取某种数据类型的某列值时, 可以指定参数为列号(JDBC 的列从 1 开始)或列名。

JDBC 使用游标来处理结果集数据, 游标可分为三种类型: TYPE_FORWARD_ONLY 游标、TYPE_SCROLL_INSENSITIVE 游标和 TYPE_SCROLL_SENSITIVE 游标。TYPE_FORWARD_ONLY 游标只能在结果集中向下滚动, 它是 JDBC 的默认游标类型。TYPE_SCROLL_INSENSITIVE 游标和 TYPE_SCROLL_SENSITIVE 游标是双向滚动游标, 区别就在于前者不会同步更新数据库中的操作, 后者会在结果集类对象中及时跟踪数据库的更新操作。这个参数可以通过指定结果集类对象的 resultSetType 参数进行修改。

JDBC 游标的打开方式不同于嵌入式 SQL, 不是显式声明而是由系统自动产生, 当结果集

刚刚生成时, 游标指向第一行数据之前, 通过 `next()`、`previous()` 等操作来移动游标获取结果集中的每一行数据。

[例 8.12] 打印教师职工号为 19950018, 教学班号为 81001-01 的学生课程评价详情。

```
String SQL "SELECT Sno, Assess, CAtype, Feedback FROM ClassAssess
WHERE Tno='19950018' AND TCno = '81001-01';
ResultSet rs = stmt.executeQuery(SQL);
while(rs.next()) {
    String Sno = rs.getString("Sno");           /* 等价于 rs.getString(1) */
    String strAssess = rs.getString("Assess");    /* 等价于 rs.getString(2) */
    Boolean bCAtype = rs.getBoolean("CAtype");   /* 等价于 rs.getBoolean(3) */
    String strFeedback = rs.getString("Feedback"); /* 等价于 rs.getString(4) */
    /* 打印课程评价详情, 注意隐藏学生的信息, 例如学生的学号 */
    System.out.printf("[ %s,%b,%s]%n", strAssess, bCAtype, strFeedback);
}
```

5. 释放资源

处理结束后, 应用程序将首先关闭结果集, 再关闭语句执行类对象, 最后释放数据库连接并与数据库服务器断开。关闭结果集使用结果集类对象提供的 `close` 方法。类似地, 关闭语句执行类对象和释放数据库连接也分别使用执行语句类对象和连接类对象的 `close` 方法。

[例 8.13] 基于 JDBC 实现 8.1.1 小节的任务 4。

为了简化描述, 输出字符串 `str` 到网页的代码使用 `System.out.println(str)` 函数代替。

如图 8.7 所示, 任务 4 涉及的实体型包括“教师”“教学班”“学生”, 实体型之间的联系包括“教师”与“教学班”之间的“讲授”, “学生”与“教学班”之间的“选课”, “教学班”“教师”“学生”三者之间的“课程评价”。

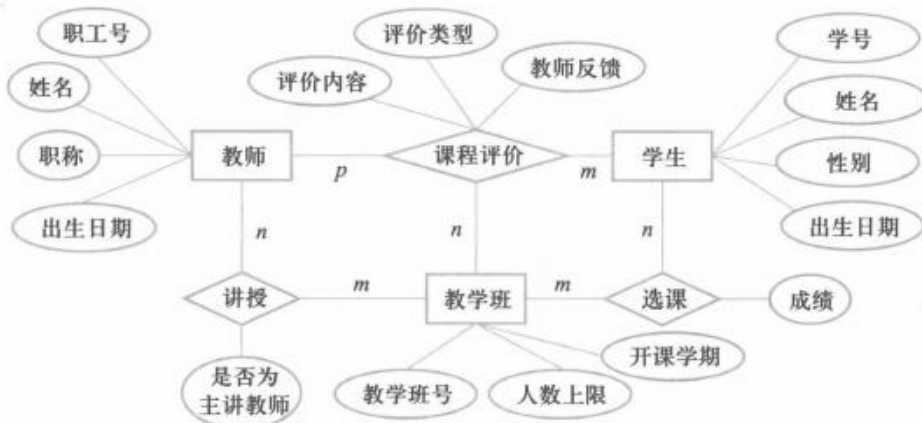


图 8.7 任务 4 涉及的实体型及其之间的联系

其中,“选课”关系模式为 SC(Sno,TCno,Grade),分别表示学生学号、教学班号、成绩。“讲授”关系模式为 Teaching(TCno,Tno,IsLeading),分别表示教学班号、职工号、是否为主讲教师。“课程评价”关系模式为 ClassAssess(Sno,Tno,TCno,Assess,CAtype,Feedback),分别表示学生学号、职工号、教学班号评价内容、评价类型、教师反馈。此外,“教师”“教学班”关系模式分别命名为 Teacher、TeachingClass。

① 数据库连接的管理。

```
public class ConnectionManager{
    static final String jdbcUrl = "jdbc:kingbase:// 192.168.0.118;54321/DB-Student";
    static final String jdbcUsername = "Info001";
    static final String jdbcPassword = "123456";
    private static Connection connection = null;
    /* 静态代码段,加载 Kingbase 数据库驱动程序 */
    static{
        Class.forName("com.kingbase.Driver");
    }
    public static Connection createConnection(){
        connection = DriverManager.getConnection(jdbcUrl,jdbcUsername,jdbcPassword);
        System.out.println("数据库连接成功...");
        return connection;
    }
    /* 释放资源 */
    public static void release(){
        if (connection != null) connection.close();
        System.out.println("释放资源成功...");
    }
}
```

② 学生浏览指定的教学班和授课教师,并输入课程评价。

```
Connection conn = ConnectionManager.createConnection();
Statement stmtnt = conn.createStatement();
/* 设置如下任务的 SQL 语句:获取学生所选的教学班及授课教师,*/
/* 假设 strSno 为该学生学号 */
String SQL4ClassAssess = "SELECT TCno,
    Tno FROM Student,Teacher,Teaching, SC
    WHERE Teacher.Tno=Teaching.Tno AND Teaching.TCno=SC.TCno AND SC.Sno="+strSno;
/* 获取执行语句 SQL4ClassAssess 的结果集 */
ResultSet rs = stmtnt.executeQuery(SQL4ClassAssess);
```

```

/* 设置插入学生课程评价的动态 SQL 语句 */
String insertSQL = "INSERT INTO ClassAssess( Sno, Tno,
    TCno, Assess, CAtype) VALUES(?,?,?,?,?)";
PreparedStatement ps = conn.prepareStatement(insertSQL);
while(rs.next()) {
    /* 阅读该学生所选的教学班及授课教师 */
    String strTno = rs.getString("Tno");
    String strTCno = rs.getString("TeachingClassNo");
    /* 对该授课教师讲授的教学班进行评价 */
    ps.setString(1, strSno);
    ps.setString(2, strTno);
    ps.setString(3, strTCno);
    ps.setString(4, "老师讲得很出色");
    /* 课程评价根据用户输入进行设置,这里仅仅是示例 */
    /* 1 为正面评价,0 为负面评价;评价类型可由用户设置 */
    ps.setBoolean(5, true);
    ps.executeUpdate();
}
rs.close();
ps.close();
stmt.close();
ConnectionManager.release();

```

③ 教师浏览教学班学生的评价意见,并针对每条评价逐一做出回复。

```

Connection conn=ConnectionManager.createConnection();
Statement stmt=conn.createStatement();
/* 获取教师指定教学班的学生课程评价。假设教师职工号为 strTno, */
/* 教学班号为 strTCno */
ResultSet rs = stmt.executeQuery("SELECT Sno, Assess, CAtype,
    Feedback FROM ClassAssess WHERE TCno
    =" + strTCno + " AND Tno
    =" + strTno);
/* 设置更新教师课程意见反馈的动态 SQL 语句 */
String updateSQL = "UPDATE ClassAssess SET Feedback
    =? WHERE Sno
    =? AND Tno
    =? AND TCno
    =?" ;

```

```

PreparedStatement ps = conn.prepareStatement(updateSQL);
while (rs.next()) {
    /* 阅读每一条课程评价 */
    String strSno = rs.getString("Sno");
    String strAssess = rs.getString("Assess");
    Boolean bCAtype = rs.getString("CAtype");
    String strFeedback = rs.getString("Feedback");
    /* 打印课程评价详情,注意隐藏学生的信息,例如学生的学号 */
    System.out.printf("[%s,%b,%s]%n",strAssess, bCAtype, strFeedback);
    /* 对该评价进行回复 */
    ps.setString(1, "感谢你的评价");          /* 反馈意见根据教师的输入进行设置 */
    ps.setString(2, strSno);                    /* 设置学生学号 */
    ps.setString(3, strTno);                    /* 设置教师职工号 */
    ps.setString(4, strTCno);                  /* 设置教学班号 */
    ps.executeUpdate();                        /* 更新教师课程意见反馈 */
}
rs.close();
ps.close();
stmt.close();
ConnectionManager.release();

```

从例 8.13 可以看出,在应用程序中直接使用 JDBC 操纵数据库,需要经过图 8.6 所示的加载驱动程序—建立与数据库的连接—执行 SQL 语句—处理结果集—释放资源的整个过程(加载驱动程序可以省略)。程序的代码管理显得比较复杂。此外,在程序的模块中需要获取数据库连接,在数据操作完毕之后再关闭数据库连接。这样对数据库进行频繁连接、开启和关闭操作,会造成数据库资源的浪费,影响数据库的性能。8.4 节基于 MVC 框架的数据库应用开发可以克服这些缺点。

* 8.4 基于 MVC 框架的数据库应用开发

MVC(model-view-controller, 模型-视图-控制器模式)是一种使用业务模型(Model)、用户结果呈现(View)以及业务模型控制器(Controller)来进行 Web 数据库应用开发的编程框架。基于 MVC 框架进行 Web 数据库应用开发,其核心思想在于功能的解耦以及功能的模块化。以本书第 7 章中介绍的“高校本科教务管理”信息系统的子系统“教师教学管理”为例进行介绍(参见图 7.30),该子系统共包括“教师”“教室”“时间片”“教学班”等实体,以及“排课”“讲授”“课程评价”等联系。其功能包括:

- ① “教师”实体的增加、删除、修改、查询。
- ② “教室”实体的增加、删除、修改、查询。
- ③ “时间片”实体的增加、删除、修改、查询。
- ④ “教学班”“教室”“时间片”三者之间“排课”联系的增加、删除、修改、查询。
- ⑤ “教学班”“教师”之间“讲授”联系的增加、删除、修改、查询。
- ⑥ “学生”“教学班”“教师”三者之间“课程评价”与回复的增加、删除、修改、查询。

基于 MVC 框架进行应用开发,需要将上述 6 个功能模块进行解耦,并按照 MVC 框架的工作流程图对每一个模块进行独立设计与实现。

图 8.8 给出了 MVC 框架的工作流程图。MVC 框架包含三部分:业务模型控制器、业务模型和用户结果呈现。每部分各自处理自己的工作,互不干扰。

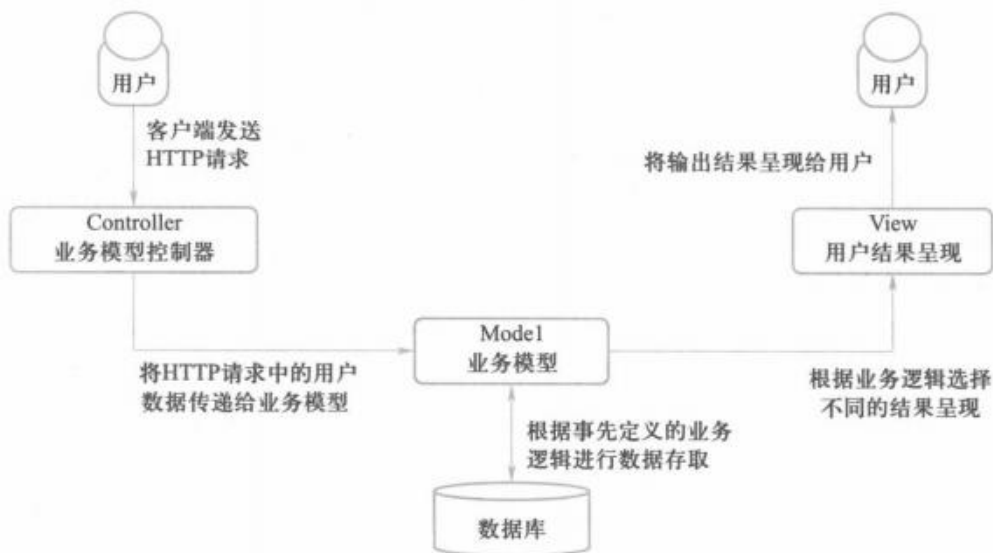


图 8.8 MVC 框架工作流程图

业务模型控制器用来接收用户通过浏览器发送的 HTTP 服务请求,该请求中包括用户需要的业务模型,以及其他可选的输入参数(可以是用户在文本框中输入的文本、下拉列表框中的数据项等,也可以是用户的操作类型)。

业务模型用来建立用户的业务逻辑。它根据业务模型控制器解析出来的用户数据,执行业务事先定义的业务逻辑,并操纵数据库存取数据。业务模型的输出独立于具体的数据格式(考虑到数据源的多样性),可以为多个用户结果呈现提供展示所需要的数据。由于业务模型可以被多个用户结果呈现重用,所以减少了代码的重复性。

用户结果呈现是用户看到并与之交互获得结果展示的界面。它根据业务模型输出的结果反馈给客户端进行呈现。

在 MVC 开发框架中,从应用程序与数据库交互的角度来看,其在应用程序中无须显式地

使用 JDBC 操纵数据库,而仅需要在业务模型中显式地建立业务与数据库之间的映射关系,由 MVC 框架自动完成应用程序与数据库之间的交互(具体可参见例 8.14)。

[例 8.14]使用 MVC 框架实现任务 4。

图 8.9 给出了使用 MVC 框架实现任务 4 的系统流程图,ClassAssessController 类对应业务模型控制器,ClassAssessModel 类对应业务模型,ClassAssessView 类对应用户结果呈现。

ClassAssessController 类分别创建了 ClassAssessService 类对象和 ClassAssessModel 类对象。ClassAssessService 类对象用来接收用户通过浏览器提交的 HTTP 请求,解析请求中用户的输入参数。以教师用户访问“教师浏览指定教学班评价与回复”功能页面为例(如图 8.9 左上角显示,用户可以通过“高校本科教务管理”信息系统主页导航到该功能页面),当教师用户浏览到学生对课程的评价并需要做出意见回复时,可以在相应的文本框中输入对学生课程评价的意见反馈,点击“回复”按钮进行意见反馈提交。Web 服务器通过解析用户的提交请求(例如图中给出的链接请求),将该 Web 请求发送给指定的业务模型控制器 ClassAssessController 类。需要注意的是,该 Web 请求中还包括请求的用户输入(即传递的变量):职工号、教学班号、学生学号、教师对学生课程评价的意见反馈。根据用户的操作请求,业务模型控制器将用户输入传递给 ClassAssessModel 类对象中的 insertFeedback 方法,把教师对指定学生课程评价的意见反馈存放到数据库中。

ClassAssessModel 类用来实现学生课程评价以及教师回复学生课程评价的应用逻辑。方法 insertFeedback 以控制器传递的学生学号 Sno、教学班号 TCno、教师职工号 Tno、教师反馈 Feedback 作为参数,将教师回复的学生评价反馈更新到数据库中。方法 getAssess 以控制器传递教学班号 TCno、教师职工号 Tno 为查询参数,查询出数据库中当前教学班所有学生的评价信息。结合方法 insertFeedback 和方法 getAssess,可以实现将教师回复指定学生课程评价的意见反馈数据更新到数据库中,并查询出数据库中当前教学班所有的最新评价数据返回 ClassAssessView 类展示给用户。方法 getTeachingClass 和方法 insertAssess 分别实现学生浏览所选教学班信息以及插入学生的课程评价信息功能。

在 MVC 开发框架中,业务模型与数据库之间的交互主要包括数据访问对象(data access object, DAO)和对象/关系数据库映射(object-relational mapping, ORM)。DAO 是将所有对数据源的访问操作抽象封装在这个类中。在应用程序中,当需要和数据源进行交互时则使用 DAO 类提供的方法进行统一访问。简单地说,DAO 就是将数据库中数据的增加、删除、修改、查询等操作封装在专门的类中,当业务模型需要与数据库进行交互时,直接调用该 DAO 类来实现 JDBC 操纵数据库的每一个步骤。

ORM 是将关系数据库中关系模式和业务模型之间做一个映射(例如关系模式 ClassAssess 和 ClassAssessModel),并建立业务模型中成员变量与关系模式属性之间的映射关系。这样,当与数据库进行交互时无须撰写复杂的 SQL 语句与数据库打交道,而只需要在应用程序中操作业务模型中的成员变量。数据库持久层框架 iBatis、myBatis、Hibernate,以及很多 PHP 框架都自带 ORM 库,在 MVC 框架中引入数据库持久层框架,通过撰写简单的 XML 配置文件即可实现关系模式和业务模型之间的映射,从而极大地简化业务模型与数据库之间的开发复杂度。

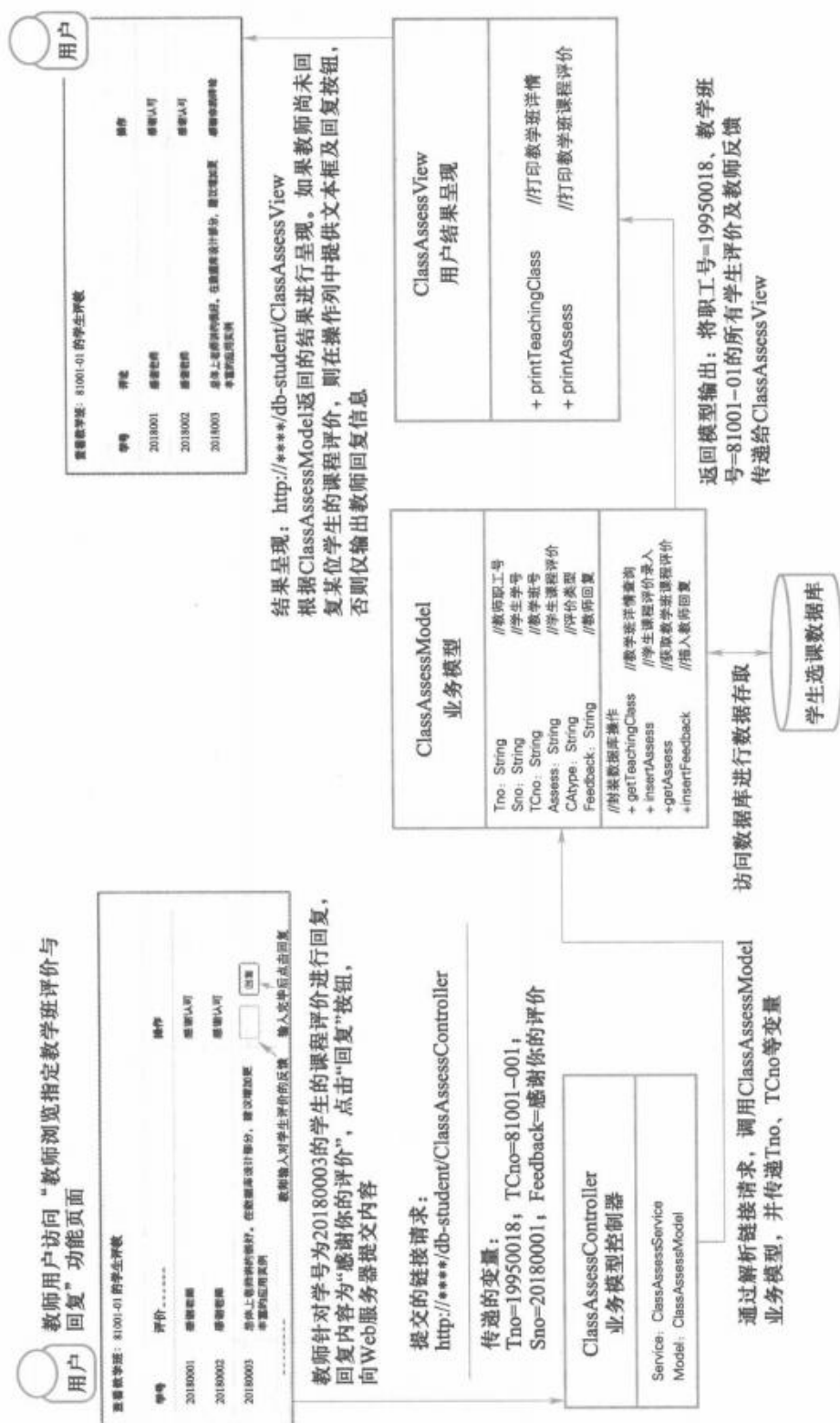


图 8.9 使用 MVC 框架实现任务 4 的系统流程图

ClassAssessView 类用来将业务模型的执行结果按照合适的方式进行结果呈现。例如, 类中的 printTeachingClass 函数实现的功能就是打印学生浏览其所选教学班及其授课教师。

由于 MVC 框架可以使用配置文件的方式支持业务模型与数据库之间的数据交互, 避免了在业务模型中显式地调用 JDBC 编程与数据库进行交互, 从而降低了应用开发的复杂度。另外, MVC 框架提供的连接池管理优化了应用程序频繁访问数据库的性能。进一步地, MVC 框架中的业务模型、用户结果呈现、业务模型控制器分离架构, 使程序更加容易开发和维护。值得一提的是, 在实际的应用开发中, 根据应用的类型、用户规模的大小, 所选用的开发模式也会有所不同。本书限于篇幅, 不再介绍其他开发模式。

本章小结

本章从两个方面讲解数据库编程技术: 一是扩展 SQL 的功能, 二是通过高级语言实现复杂应用。

本章讲解了扩展 SQL 能力的技术途径, 具体包括引入新的 SQL 子句、引入新的内置函数、引入 PL/SQL 以及存储过程/存储函数等技术。存储过程和存储函数经编译和优化后存储在数据库服务器中, 运行效率高, 客户机和服务器之间的通信量小, 可以集中控制管理, 因此被广泛使用。

SQL 与高级语言具有不同的数据处理方式。SQL 是面向集合的, 而高级语言是面向记录的。游标就是用来协调这两种不同的处理方式的机制。要掌握游标的概念, 学会用游标来编写实际的应用程序。

本章还阐述了 JDBC 的工作原理和工作流程, 即在 Java 程序中如何访问数据库中的数据。使用 JDBC 编写的应用程序可移植性好, 能同时访问不同的数据库, 共享多个数据资源。本章最后讲解了基于 MVC 框架的开发方式, 在实际的项目开发中, 采用开发框架的方式可以提高应用开发的效率。

习 题 8

1. 假定一门课程存在多门直接先修课程, 使用 WITH RECURSIVE 子句查找“数据库系统概论”课程的所有先修课课程号和课程名称。
2. 对“学生选课”数据库编写存储过程, 完成下述功能:
 - ① 统计“离散数学”课程的成绩分布情况, 即按照各分数段统计人数。
 - ② 统计任意一门课程的平均成绩。
 - ③ 将学生选课成绩从百分制改为等级制(即 A、B、C、D、E)。
3. 设有产品表的关系模式 Product(PID, PName, Price, Amount, DateOfManufacture), 描述的是产品编号、产品名称、单价、数量、生产日期, 其中主码为产品编号与生产日期。请基于某一种关系数据库管理系统, 使用数据定义语言完成 Product 关系模式的创建, 并使用存储过程或存储函数生成不少于 1 000 条产品记录。
4. 使用 JDBC 编写应用程序, 实现对异构数据库中的数据进行迁移。要求:
 - ① 配置两种不同的数据源, 编写程序连接两种不同关系数据库管理系统的数据源。
 - ② 分别在上述两种关系数据库管理系统中完成第 3 题产品关系模式创建及产品记录的生成。

③ 使用 JDBC 编写应用程序，将一种关系数据库管理系统产品表的数据迁移到另一种关系数据库管理系统的产品表，进行追加。

第 8 章实验 数据库编程与大作业

实验 8.1 存储过程实验。掌握数据库 PL/SQL 编程语言，以及数据库存储过程的设计和使用方法，包括存储过程定义、存储过程运行、存储过程更名、存储过程删除、存储过程的参数传递。掌握 PL/SQL 编程语言和编程规范，规范设计存储过程。

实验 8.2 自定义函数。掌握数据库 PL/SQL 编程语言以及数据库自定义函数的设计和使用方法，包括自定义函数定义、自定义函数运行、自定义函数更名、自定义函数删除、自定义函数的参数传递。掌握 PL/SQL 和编程规范，规范设计自定义函数。

实验 8.3 游标实验。掌握 PL/SQL 游标的设计、定义和使用方法，理解 PL/SQL 游标按行操作和 SQL 按结果集操作的区别和联系。掌握各种类型游标的特点、区别与联系。

实验 8.4 基于 JDBC 访问数据库的应用开发。掌握基于 JDBC 驱动的数据库应用开发方法、基于 JDBC 驱动的数据库连接方法，实现数据库数据操纵等应用开发常见功能。

实验 8.5 数据库设计与应用开发大作业。掌握综合运用数据库原理、方法和技术进行数据库应用系统分析、设计和开发的能力。能够针对某部门或单位的应用需求，通过系统分析，从数据库数据和应用系统功能两方面进行综合设计，实现一个完整的数据库应用系统。

参考文献 8

- [1] RDBMS 用户使用手册。
- [2] RDBMS 编程手册。
- [3] 关于 Java 和 JDBC，可参阅 Oracle 官网上的 Java Tutorials 文档，也可从 MSDN(Microsoft Developer Network)上获得相关的资料。
- [4] 王珊，张俊．数据库系统概论(第 5 版)习题解析与实验指导[M]．北京：高等教育出版社，2014。

本章知识点讲解微视频：



SQL 语言表达能力的限制



过程化 SQL 例子



存储过程的使用场景



JDBC 架构与工作原理

第三篇

系 统 篇

本篇讲解数据库管理系统中查询处理和事务管理的基本概念和基础知识。

本篇包括5章。

第9章关系数据库存储管理。存储管理是数据库管理系统的重要职责之一，它与查询优化以及数据库物理设计都密切相关。本章介绍关系数据库的数据组织方法和5种常用的索引结构。

第10章关系查询处理和查询优化，讲解关系数据库管理系统查询处理的基本步骤，实现选择操作和连接操作的算法，查询优化的概念、方法和技术，以及查询计划的执行方法。

第11章数据库恢复技术，讲解事务的基本概念和性质，数据库系统遇到故障后进行恢复的技术和方法，以及提高故障恢复效率的检查点技术和数据库镜像技术。

第12章并发控制，介绍并发操作可能造成数据不一致的问题，以及事务的4种隔离级别；讲解并发控制的基本概念和最常用的封锁技术；讨论并发调度的可串行性，以及如何用封锁技术保证并发事务的可串行性。

*第13章数据库管理系统，阐述数据库管理系统的基本功能、系统结构及主要的实现技术。本章可以作为选读内容，为读者进一步学习数据库内核实现技术打下基础。

第9章

关系数据库存储管理

本章讨论关系数据库的数据组织和存储管理。我们将重点介绍基于磁盘的数据库的组织与存储，有关内存数据库的数据组织见新技术篇的第17章。

本章导读

存储管理是数据库管理系统(DBMS)的重要职责之一。它与查询优化以及数据库物理设计都密切相关。

本章介绍关系数据库的数据组织和索引结构。在9.1节数据组织中主要讨论数据库的逻辑与物理组织方式，重点介绍记录的表示、记录如何在块中组织存储以及如何组织关系表的存放；在9.2节索引结构中重点介绍5类索引，包括顺序表索引、辅助索引、B+树索引、哈希索引和位图索引。

本章的难点是理解数据库逻辑组织与物理组织之间的对应关系，了解关系表的几种存放方式，深入理解索引的作用，掌握B+树索引和哈希索引的组织方式和查找方式。

在第一篇基础篇中我们已经知道，关系数据库的一个重要特点是存储结构和存取路径对用户隐蔽，用户访问数据库时，只需要用SQL语句表达自己需要哪些信息，比如“查询数据库系统概论课程的选课人数”，而不需要像过程化语言那样具体指定怎么去查找。将用户的查询请求转化成查询执行计划，是数据库查询优化的一个重要工作，而了解关系数据库的数据组织与存储管理，能帮助我们更好地理解关系数据库查询优化的原理。同时，了解数据组织与存储管理，也能帮助我们在应用开发中更好地进行数据库物理设计。

9.1 数据组织

数据库是大量数据的有结构的综合性集合。如何将这样一个庞大的数据集合以最优的形式组织起来存放在外存上，是一个非常重要的问题。所谓“优”应包括两方面：一是存储效率高，节省存储空间；二是存取效率高，访问速度快，代价小。本节主要介绍数据库的逻辑组织方式和物理存储方式。